

A.E.D. Individual 1

Guillermo Román

guillermo.roman@upm.es

Lars-Åke Fredlund

lfredlund@fi.upm.es

Manuel Carro

mcarro@fi.upm.es

Julio García

juliomanuel.garcia@upm.es

Tonghong Li

tonghong@fi.upm.es

Francisco de Borja Lozano

francisco.lozano@upm.es

Normas.

- ▶ Fechas de entrega y penalización:

Hasta el Martes 22 de septiembre, 16:00 horas	0
Hasta el Miércoles 23 de septiembre, 16:00 horas	20 %
Hasta el Jueves 24 de septiembre, 16:00 horas	40 %
Después la puntuación máxima será 0	
- ▶ Usad las horas de tutoría para preguntar sobre programación – son oportunidades excelentes para aprender
- ▶ Se comprobará plagio y el uso no permitido de IA's, y se actuará sobre los casos detectados

IDEs y uso de IA

- ▶ Para realizar el proyecto se puede usar diferentes IDEs (Eclipse, Visual Studio, IntelliJ) o la línea de comandos (javac y java). Notad que los profesores pueden ofrecer ayuda en como configurar Eclipse, pero no necesariamente sobre otros IDEs.
- ▶ **No está permitido el uso de IA (LLMs) para escribir código, ni como apoyo para buscar soluciones algorítmicas.** Se permite el uso de IA para resolver dudas sobre construcciones de Java.
- ▶ Os recomendamos, para mejorar vuestro aprendizaje, configurar el IDE para que no os ofrezca ayuda de codificación en modo IA.

Entrega

- ▶ Todos los ejercicios de laboratorio se deben entregar a través de

`http://deliverit.fi.upm.es`

- ▶ El/los fichero(s) que hay que subir es/son `Utils.java`.
- ▶ La clase debe estar en el paquete `aed.individual1`
- ▶ La documentación de la API de `aedlib.jar` está disponible en

`http://costa.ls.fi.upm.es/teaching/aed/docs/aedlib/`

Tarea 1: Borrar elementos repetidos en una lista indexada

- ▶ Se pide implementar el siguiente método dentro la clase Utils:

```
static <E> void eliminarRepetidos(IndexedList<E> l)
```

- ▶ Recibe una lista indexada l, y borra de la lista todos los elementos que ocurren más que una vez dentro l
- ▶ El orden de elementos en l debe preservarse
- ▶ Las listas indexadas están disponibles en la librería aedlib.jar
- ▶ La documentación de su API está en <http://costa.ls.fi.upm.es/entrega/aed/docs/aedlib/es/upm/aedlib/indexedlist/IndexedList.html>

Ejemplos

<code>eliminarRepetidos([])</code>	-->	<code>l = []</code>
<code>eliminarRepetidos([1])</code>	-->	<code>l = [1]</code>
<code>eliminarRepetidos([1,4,3])</code>	-->	<code>l = [1,4,3]</code>
<code>eliminarRepetidos([1,4,3,4,3])</code>	-->	<code>l = [1]</code>
<code>eliminarRepetidos([3,1,4,4,3,4,2,3])</code>	-->	<code>l = [1,2]</code>

Tarea 2: Devolver los elementos repetidos

- ▶ Se pide implementar el siguiente método dentro la clase Utils:

```
static <E> IndexedList<E>  
    elementosRepetidos(IndexedList<E> l)
```

- ▶ Recibe una lista indexada, y devuelve una **nueva** lista (resultado) donde solo estan los elementos repetidos dentro de l. Notad que resultado solo debe contener un copia de cada elemento
- ▶ El orden de elementos en l debe preservarse en el resultado
- ▶ No se debe modificar la lista de entrada l
- ▶ Para crear una lista indexada se puede usar

```
IndexedList<E> resultado = new ArrayIndexedList<E>();
```

Ejemplos

<code>elementosRepetidos([])</code>	<code>--></code>	<code>[]</code>
<code>elementosRepetidos([1])</code>	<code>--></code>	<code>[]</code>
<code>elementosRepetidos([1,4,3])</code>	<code>--></code>	<code>[]</code>
<code>elementosRepetidos([1,1])</code>	<code>--></code>	<code>[1]</code>
<code>elementosRepetidos([1,1,2,1])</code>	<code>--></code>	<code>[1]</code>
<code>elementosRepetidos([1,4,3,3,4,4])</code>	<code>--></code>	<code>[4,3]</code>
<code>elementosRepetidos([1,4,4,3,4,2,3])</code>	<code>--></code>	<code>[4,3]</code>

Notas importantes

- ▶ El valor de los parametros 1 nunca serán `null` y nunca contendrá elementos `null`
- ▶ **No se debe modificar la estructura de datos recibida como parámetro**, excepto cuando explícitamente lo permite en el enunciado (p.e., para `eliminarRepetidos`).
- ▶ El proyecto debe compilar sin errores y debe cumplirse la especificación de los métodos a completar.
- ▶ Debe ejecutar `TesterInd1` correctamente sin mensajes de error.
- ▶ **Nota:** una ejecución sin mensajes de error **no** significa que el método sea correcto (es decir, que funcione bien para cualquier entrada posible).